Rapport Projet Web

JENY JEYARAJ Jeen (22519719) & DORCAL Doralie (22208270)
M1 Cybersécurité et E-santé

12 janvier 2026

Table des matières

Introduction **1**
Structure du dossier 1
Présentation du projet 1
Setup 2
Lancer l'application 2
I) Création des deux services **3**
Frontend 3
Backend 3
II) Gateway, ingress, nginx, service mesh **4**
Ingress et nginx 4
Istio 5
III) MySQL : une BDD en local **5**
IV) Sécuriser le cluster **6**
Conclusion **7**

Introduction

Structure du dossier

```
Microservices-and-Security-Kubernetes/
        |----- frontend/
        |         |----- app.py
        |         |----- requirements.txt
        |         |----- Dockerfile
        |         └--- templates
        |                   └--- index.html
        |---- backend/
        |         |----- app.py
        |         |----- requirements.txt
        |         └--- Dockerfile
        |---- kube/
        |         |----- backend-deploy.yml
        |         |----- backend-service.yml
        |         |----- frontend-deploy.yml
        |         |----- frontend-service.yml
        |         |----- ingress.yml
        |         |----- istio-gateway.yml
        |         |----- istio-security.yml
        |         |----- mysql-deployment.yml
        |         |----- mysql-pvc.yml
        |         |----- mysql-service.yml
        |         |----- network-policy.yml
        |         |----- rbac.yml
        |         └--- secrets.yml
        └--- README.md
```

Présentation du projet

Ce projet consiste à concevoir, déployer et sécuriser une application basée sur une architecture en microservices. L'application est divisée en deux composants principaux développés en Python : un frontend interactif pour l'utilisateur, et un backend chargé de la logique métier. L'ensemble s'appuie sur une base de données locale MySQL pour le stockage et la gestion des données.

Pour garantir un déploiement fiable, flexible et reproductible, chaque composant est conteneurisé à l'aide de Docker, puis orchestré au sein d'un cluster Kubernetes.

L'accès à l'application depuis l'extérieur est géré par une passerelle Ingress Nginx, qui se charge de diriger le trafic vers les bons conteneurs. En parallèle, un composant de type Service Mesh (Istio) est déployé pour contrôler et optimiser la communication interne entre le frontend et le backend.

Cette architecture assure le bon fonctionnement de l'application. Or, nous pouvons aller plus loin en durcissant la sécurité de l'ensemble de l'infrastructure.

Notre application est un recueil de notes; notre petit journal intime. Nous pouvons écrire des notes et les enregistrer. Ces-dernières seront datées et resteront stockées dans la BDD en tant que données persistantes.

Setup

Nous avons commencé par créer un dossier Microservices-and-Security-Kubernetes, dans lequel nous allons faire notre projet.

Lien vers le Github du projet :
https://github.com/JeenJJJ/Microservices-and-Security-Kubernetes
```
git clone https://github.com/JeenJJJ/Microservices-and-Security-Kubernetes.git
```

Dans le terminal, nous nous sommes d'abord connectées au Docker :
```
docker login
```

Puis, nous avons ouvert la page donnée dans le terminal, afin de saisir le OTP donné et se connecter.

Lancer l'application

Dans le terminal :
```
minikube start
minikube addons enable ingress
minikube addons enable istio-provisioner
minikube addons enable istio
minikube addons enable ingress-dns
kubectl label namespace default istio-injection=enabled
kubectl apply -f kube/
minikube tunnel
```

Dans votre browser préféré : http://journalintime.info

I) Création des deux services

Frontend

Le frontend est comme une vitrine, car il affiche les informations à l'utilisateur et capte ses actions.

Nous avons codé une mini application en python “app.py” et son Dockerfile.

Puis, nous avons créé et tagué une image Docker :
`docker build -t doralie/journal-front:1.0 .`

Ensuite, nous avons publié l’image sur Docker Hub :
`docker push doralie/journal-front:1.0`

Et enfin, nous avons créé un déploiement Kubernetes :
`kubectl apply -f kube/frontend-deploy.yml`

Et enfin, nous avons créé un service Kubernetes :
`kubectl apply -f kube/frontend-service.yml`

Backend

Le backend reçoit les demandes du frontend, fait les calculs et va chercher les informations.

Nous avons codé une mini application en python “app.py” et son Dockerfile.

Puis, nous avons créé et tagué une image Docker :
`docker build -t doralie/journal-back:1.0 .`

Ensuite, nous avons publié l’image sur Docker Hub :
`docker push doralie/journal-back:1.0`

Et enfin, nous avons créé un déploiement Kubernetes :
`kubectl apply -f kube/backend-deploy.yml`

Et enfin, nous avons créé un service Kubernetes :
`kubectl apply -f kube/backend-service.yml`

II) Gateway, ingress, nginx, service mesh

Ingress et nginx

Ceci reçoit les requêtes venant d'Internet et les dirige vers notre frontend.
Ingress : https://github.com/charroux/kubernetes-minikube/blob/main/ingress.yml

Nous avons repris le code et changé le nom en “journal-ingress” et le hostname en “journalintime.info”.

Dans le terminal :
minikube addons enable ingress
kubectl get pods -n ingress-nginx

```
NAME                                        READY   STATUS      RESTARTS      AGE
ingress-nginx-admission-create-6kssn        0/1     Completed   0             5d5h
ingress-nginx-admission-patch-75fcp         0/1     Completed   0             5d5h
ingress-nginx-controller-9cc49f96f-wmmzx    1/1     Running     3 (63s ago)   5d5h
```

kubectl apply -f kube/ingress.yml
kubectl get ingress

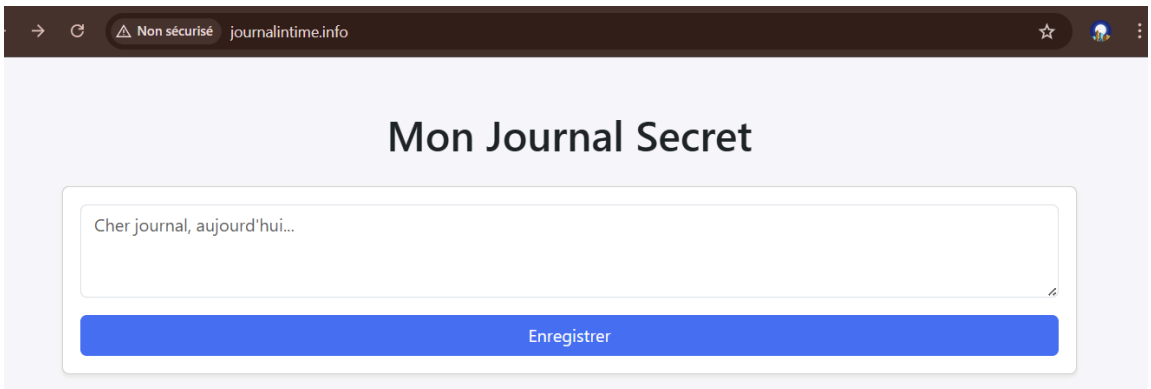
```
NAME             CLASS   HOSTS                ADDRESS        PORTS   AGE
journal-ingress  nginx   journalintime.info   192.168.49.2   80      45m
```

Nous travaillons sur Windows, donc dans c:\windows\system32\drivers\etc\hosts, on ajoute la ligne suivante :
127.0.0.1 journalintime.info

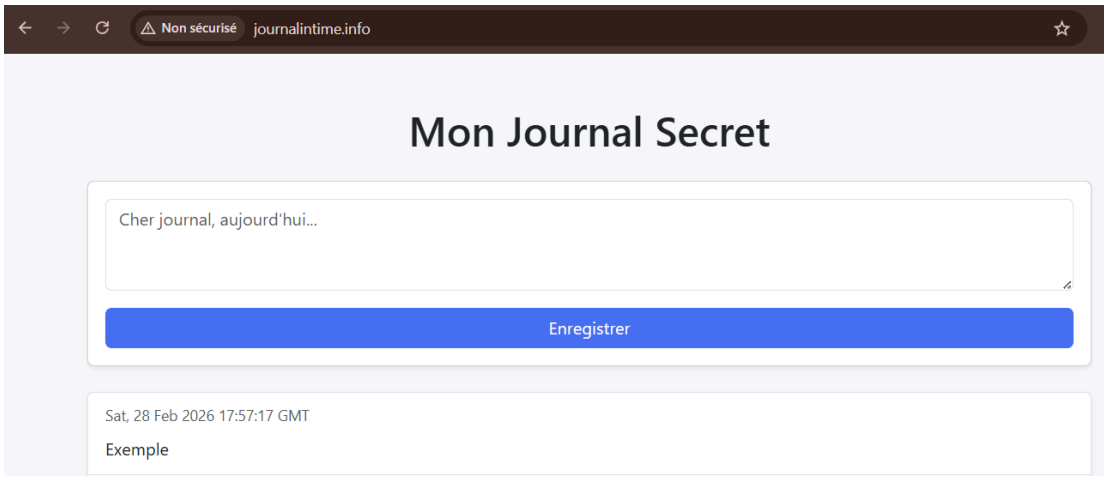
minikube addons enable ingress-dns
minikube tunnel

Puis, nous pouvons accéder à notre site via : http://journalintime.info

Nous voyons que le site fonctionne :



Nous pouvons maintenant nous amuser à ajouter des notes à notre journal intime :



Istio

Le service mesh gère la communication entre les deux microservices créés et surveille le trafic interne, afin de s'assurer que tout circule bien.

Deux nouveaux fichiers :

- Définir la gateway Istio et les routes vers nos deux services :
  kubectl apply -f kube/istio-gateway.yml
- Activer la sécurité Istio (par exemple le mTLS) entre nos services :
- kubectl apply -f kube/istio-security.yml

Dans les fichiers de déploiement pour le frontend et le backend, nous avons ajouté une ligne : sidecar.istio.io/inject: "true"

III) MySQL : une BDD en local

Nous avons aussi besoin d'une base de données locale où le backend va stocker et récupérer les données.

MySQL : https://github.com/charroux/noops/tree/main/mysql
Nous l'avons utilisé et adapté à notre projet.

Lancer le conteneur MySQL dans Kubernetes :
kubectl apply -f kube/mysql-deployment.yml

Réserver un stockage persistant pour que MySQL garde les données :
kubectl apply -f kube/mysql-pvc.yml

Permet aux autres services du cluster d'accéder à MySQL via un nom stable :
`kubectl apply -f kube/mysql-service.yml`

Contenir les identifiants sensibles comme le mot de passe MySQL :
`kubectl apply -f kube/secrets.yml`

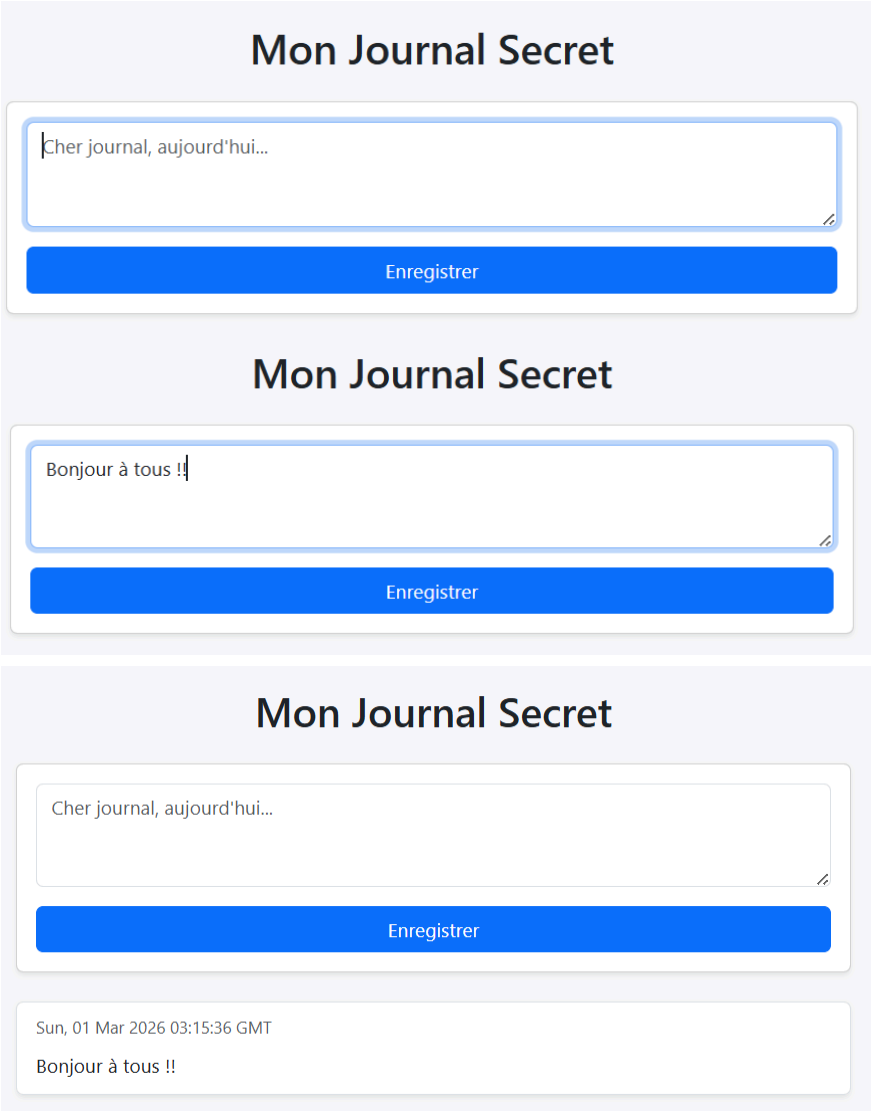
IV) Sécuriser le cluster

Afin d'assurer la sécurité de l'ensemble de l'infrastructure, nous avons mis en place plusieurs moyens :

- **Registry image sécurisé (Docker Hub) :**
  En effet, Docker Hub effectue un "Vulnerability Scanning" (scan de failles) sur nos deux images "doralie/journal-back" et "doralie/journal-front" pour s'assurer qu'elles ne contiennent pas de virus ou de librairies périmées.
- **Sécurité Proxy (Service Mesh, Istio) :**
  Dans nos fichiers de déploiement, nous avons ajouté l'annotation qui force l'injection du proxy Envoy, c'est-à-dire le "sidecar", devant nos applications.
- **Cryptage des échanges (Mutual TLS) :**
  Pour que les microservices se parlent de manière cryptée, nous avons un fichier nommé "istio-security.yml" qui force le cryptage mTLS entre nos deux services. Ainsi, personne ne peut interférer entre les messages du frontend et du backend.
- **3 outils de contrôle de la sécurité Kubernetes :**
  - RBAC (Role-based access control) : un mécanisme de contrôle d'accès qui définit les rôles et les autorisations de chaque utilisateur
  - Network Policies : un pare-feu interne qui n'autorise que le backend à parler à la base de données. Ceci empêche qu'un pirate ne vole nos données s'il parvient à s'introduire par le frontend.
  - Secrets : définit le mot de passe qui sera utilisé lors de la connexion au serveur MySQL.

Conclusion

En conclusion, ce projet nous a permis de suivre l'ensemble des étapes clés du cycle de vie d'une application, de sa conception à sa sécurisation.

Nous avons développé deux microservices en python, le frontend et le backend, en utilisant une base de données MySQL locale. D'une part, Docker et Kubernetes nous ont assuré un déploiement fiable et reproductible. D'autre part, l'intégration d'Ingress Nginx et d'Istio a garanti un accès sécurisé et une communication interne fluide entre les services. De plus, sécuriser le cluster nous a montré l'importance de protéger les données et le fonctionnement de l'application.

Ainsi, ce projet illustre comment combiner efficacité, modularité et sécurité dans une architecture moderne, tout en offrant une application fonctionnelle et interactive : notre propre journal intime numérique.